

Exploration Strategies in Deep Q-Learning using Super Mario

Darby Waters

Northeastern University
Khoury College of Computer Sciences
360 Huntington Ave, Boston, MA 02115

Abstract

Exploration strategy is a critical design choice in reinforcement learning. It directly impacts an agent's ability to learn an optimal policy. In this paper, we evaluate four exploration strategies using Deep Q Network in the Super Mario Bros environment. The four strategies used are E-greedy, Softmax, Noisy Networks, and a custom heuristic approach. Each agent is trained for two million time steps, and most strategies use epsilon or temperature scheduling, with the exception of Noisy Networks. The results show that the heuristic strategy performed the best (177.64 average return after training), outperforming E-greedy (173.94), Softmax (163.92), and Noisy Networks (151.21).

1 Introduction

The exploration vs exploitation tradeoff is a common research focus in reinforcement learning. The balance between exploration and exploitation can have a significant impact on the final average return. In reinforcement learning, actions can be referred to as greedy and non-greedy. Agents keep track of which actions are currently performing the best during learning. The action that is currently doing the best is called a greedy action. When a greedy action is taken, that is exploiting. However, when a non-greedy action is taken, it is considered exploration. (Sutton and Barto 2018) In this paper, we will evaluate how four different exploration strategies perform when applied to a Deep Q Network algorithm in a Super Mario Bros environment.

In reinforcement learning, video games are an excellent research environment. They offer visual environments that are easy to see progression, clear reward structures, and easy-to-understand goals. For this project, Super Mario Bros was selected for the same reasons listed previously. Super Mario Bros is also commonly used as an environment in reinforcement learning, especially within applications of Deep Q Network.

2 Background

Markov Decision Process

This project utilizes a Markov decision process (MDP). An MDP can be defined as "a classical formalization of sequen-

tial decision making, where actions influence not just immediate rewards, but also subsequent situations, or states, and through those future rewards". (Sutton and Barto 2018) The MDP for this project is as follows:

- State Space: $s \in \mathbb{R}^{(4 \times 84 \times 84)}$
- Action Space: 'SIMPLE_MOVEMENTS'
 - This includes: 'NOOP', 'RIGHT', 'RIGHT+A', 'RIGHT+B', 'RIGHT+A+B', 'A', and 'LEFT'
- Reward Function: $r = v + c + d$
 - v is the difference in agent x values between the states
 - c is the difference in the game clock between frames
 - d is a death penalty of -15 if the agent dies during an episode

The environment is an 84 x 84 pixel playable area with 4 frames stacked together. The action space is provided by the Super Mario Bros Gymnasium environment. (Kauten 2018a) There are three action spaces to choose from: 'RIGHT_ONLY', 'SIMPLE_MOVEMENTS', and 'COMPLEX_MOVEMENTS'. Where 'A' is jump, 'B' is sprint, and 'NOOP' is no movement. 'SIMPLE_MOVEMENT' was chosen for this paper to give a simplistic action space, while not going too small.

Q-Learning

The reinforcement algorithm used, Deep Q Networks, will utilize Q-Learning. Q-learning learns the Q-value of a specific state-action pair. The q-value is a value given to a state-action pair that corresponds to the expected reward when taking action a in state s . (Sutton and Barto 2018). The specific update equation is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

In this equation, α is the learning rate, γ is the discount factor, and r is the reward.

Deep Q Network

Deep Q Network (DQN) combines Q-learning and neural networks. DQN utilizes a replay buffer and target network to estimate Q-values without having to store every one. The replay buffer is used to store up to a certain number of past transitions. The target network is used to compute targets in

order to minimize loss. Loss is calculated from the following loss function (Mnih and others 2015):

$$L = \mathbb{E}[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)) \Delta Q(s, a; \theta)]$$

Convolutional Neural Network

This paper also utilizes a Convolutional Neural Network (CNN). A CNN is a neural network designed for environments with data that comes in multiple arrays (Macas, Wu, and Fuertes 2022). This is especially useful in video game environments.

3 Related Work

A paper titled 'Playing Atari with Deep Reinforcement Learning' was the first paper to successfully apply Deep Q-Learning to video games. (Mnih and others 2015) In 'Playing Atari with Deep Reinforcement Learning', e-greedy was the exploration choice. Thus, leading to this as the baseline in this paper. Also in the paper by Mnih et al, the action space was scaled in the same manner as what was done in this paper.

Another exploration strategy that was considered but not used is Curiosity-Driven exploration. This strategy encourages the agent to visit "novel" states. These "novel" states are found by calculating novelty. According to the paper 'Curiosity-driven Exploration by Self-supervised Prediction', this novelty can be calculated using a statistical model of the distribution of the environmental states, and measuring uncertainty requires building a model of environmental dynamics that predicts the next state given the current state and the action executed at time t . (Pathak et al. 2017) This strategy was ultimately not included in this paper because it is less effective when used for dense rewards.

A second exploration strategy that was not used in this paper is Upper Confidence Bound exploration. This exploration strategy calculates an exploration bonus based on how often the states are visited. (Sutton and Barto 2018) This strategy was not used because it does not perform well in large state spaces.

4 Project Description

Environment

As mentioned previously, the environment is Super Mario Bros. The specific environment comes from the Gymnasium package gym-super-mario-bros. (Kauten 2018a) The level that the agents are trained on is Super-Mario-Bros-v0. This is able to be run using nes-py, which is an NES emulator for gymnasium packages. (Kauten 2018b)

The environment is originally an RGB 240x256 pixel playable area. From this original environment, there is a custom grayscale wrapper applied, the playable area is then resized to 84x84 pixels, and finally, four frames are stacked together. The grayscale is applied to reduce computation. The playable area is resized to increase training speeds. Finally, the frames are stacked to give velocity information.

The action space is provided by the gymnasium package. There are three available action spaces to use 'RIGHT_ONLY', 'SIMPLE.MOVEMENTS', and

'COMPLEX.MOVEMENTS'. For this project, 'SIMPLE.MOVEMENTS' was selected. This action space gives seven usable actions; these actions include 'NOOP', 'RIGHT', 'RIGHT+A', 'RIGHT+B', 'RIGHT+A+B', 'A', and 'LEFT'. Where 'A' is jump, 'B' is sprint, and 'NOOP' is no movement.

Network Architecture

The network architecture used is a Convolutional Neural Network. (Mnih and others 2015) The network takes in the observation space (84x84x4), and pixel values are normalized by dividing by 255. There is 3 convolutional layers and 2 fully connected layers for all agents, except for Noisy Networks, where there is one change mentioned later. The first convolutional layer applies 32 filters of size 8x8 with a stride of 4, followed by a ReLU activation. The second applies 64 filters of size 4x4 with a stride of 2, followed by ReLU. The third applies 64 filters of size 3x3 with a stride of 1, followed by ReLU. The output is then flattened and passed through two fully connected layers: a hidden layer of 512 with ReLU activation, and an output layer with 7 corresponding to the 7 possible actions.

DQN Algorithm

The DQN algorithm used can be represented by the following pseudo-code:

Algorithm 1 DQN

```

1: for  $t = 1$  to 2,000,000 do
2:    $\epsilon \leftarrow \text{schedule}(t; 1.0 \rightarrow 0.01 \text{ over } 1,000,000)$ 
3:    $a \leftarrow \text{get\_action}()$  ▷ changes for each agent
4:    $s', r, done \leftarrow \text{step}(a)$ 
5:   store  $(s, a, r, s', done)$  in replay buffer
6:    $s \leftarrow s'$ 
7:   if  $done$  then
8:     reset env
9:   end if
10:  if  $t > 10,000$  then
11:    if  $t \bmod 4 = 0$  then
12:      sample batch  $(s, a, r, s', done) \sim \text{replay}$ 
13:      buffer
14:       $q \leftarrow Q(s, a)$ 
15:       $target \leftarrow r + \gamma(1 - done) \max_{a'} Q(s', a')$ 
16:      update  $\theta$  using Adam optimizer
17:    end if
18:    if  $t \bmod 2000 = 0$  then
19:       $\theta^- \leftarrow \theta$ 
20:    end if
21:  end for
```

When running the algorithm, the following parameters were selected across all agents:

- Max Time Steps: 2,000,000
- Capacity of Replay Buffer: 100,000
- Learning rate: 1e-4

- Learning is delayed until after 10,000 time steps
- Behavior policy is updated every 4 time steps
- Target network is updated every 2,000 time steps
- Batch size: 64
- Gamma: 0.99
- Adam Optimizer used

E-greedy

The `get_action` function for the e-greedy agent is as follows:

Algorithm 2 E-greedy: `get_action()`

```

1: if  $random < \epsilon$  then
2:   return random action
3: else
4:    $q\_values \leftarrow Q\theta(s)$ 
5:   return  $\operatorname{argmax} q\_values$ 
6: end if

```

The agent was chosen to act as the baseline for all other agents. The linear scheduling was ϵ starting at 1.0 and decaying to 0.01 over 1,000,000 time steps.

Softmax

Algorithm 3 Softmax: `get_action()`

```

1:  $q\_values \leftarrow Q\theta(s)$ 
2:  $probs[a] \leftarrow \exp(q\_values/T) / \sum \exp(q\_values[*])/T$ 
3: return sample action from probs

```

This agent uses temperature scheduling as its action selection; the formula for that is the following:

$$\pi(a|s) = \exp(Q(s, a)/T) / \sum \exp(Q(s, a')/T)$$

Where T is temperature. The temperature scheduling was temperature starting at 5.0 and decaying to 0.1 over 1,000,000 time steps.

Heuristic

Algorithm 4 Heuristic: `get_action()`

```

1: if  $random < \epsilon$  then
2:    $better\_actions \leftarrow 1, 2, 3, 4$ 
3:    $worse\_actions \leftarrow 0, 5, 6$ 
4:   if  $random < 0.8$  then
5:     return random action from better actions
6:   else
7:     return random action from worse actions
8:   end if
9: else
10:   $q\_values \leftarrow Q\theta(s)$ 
11:  return  $\operatorname{argmax} q\_values$ 
12: end if

```

This agent uses the same epsilon scheduling as e-greedy and a very similar overall structure. The difference is the random action selection. Here, the agent chooses actions that move right 80% of the time and left actions 20% of the time.

Noisy Network

Algorithm 5 Noisy Network: `get_action()`

```

1:  $q\_values \leftarrow Q(s)$   $\triangleright$  noise applied automatically in forward pass
2: return  $\operatorname{argmax} q\_values$ 

```

For Noisy Network, the network architecture slightly changes. The CNN is modified so that instead of using 2 fully connected layers, there is now a custom Noisy Linear layer that allows exploration from the network weights rather than epsilon. The σ is set to 0.1. Since the exploration is achieved through the network weights instead of epsilon, there is no linear scheduling applied to this agent. (Fortunato and others 2017)

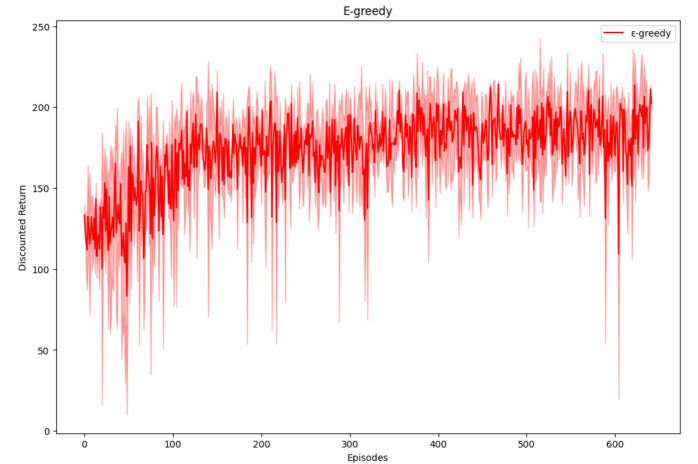
5 Experiments

Setup

All agents were run on an NVIDIA RTX 3060 GPU, each agent taking about 2.5 hours per seed. The agents were run on 3 seeds each, including 1234, 5678, and 9012. The total training time was around 30 hours. The training was run headless (without visuals) for faster training speeds. The returns for each seed and agent were saved in an npy file after every episode.

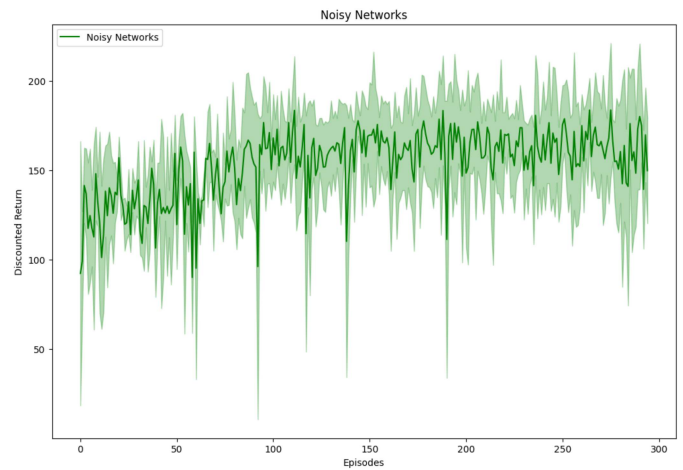
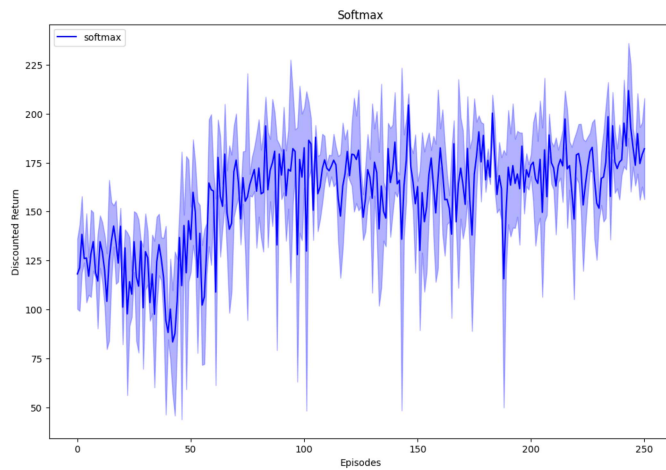
E-greedy results

E-greedy starts with a return of around 130 and converges around 190. The average return across all episodes across all seeds was 173.94. The learning curve plateaus around episode 400. The learning curve can be seen below:



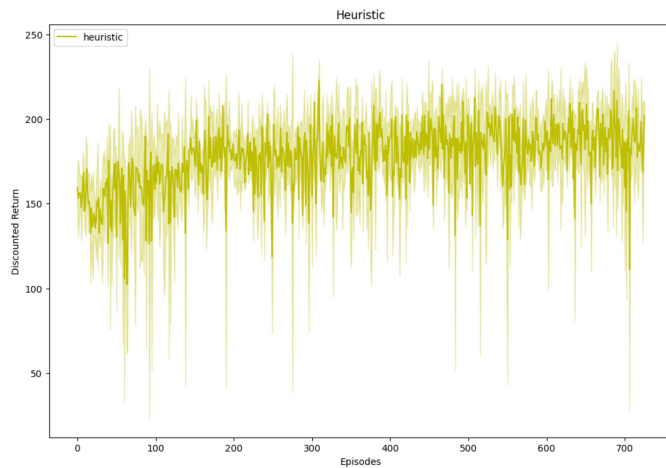
Softmax results

Softmax starts with a return of around 125 and converges around 180. The average return across all episodes across all seeds was 163.92. The agent in Softmax took longer in each episode compared to e-greedy, suggesting that the agent was surviving longer in each episode. The learning curve can be seen below:



Heuristic results

The heuristic method starts with a return of around 150 and converges around 190. The average return across all episodes across all seeds was 177.64. The learning curve plateaus similarly to e-greedy, suggesting a similar policy. The learning curve can be seen below:



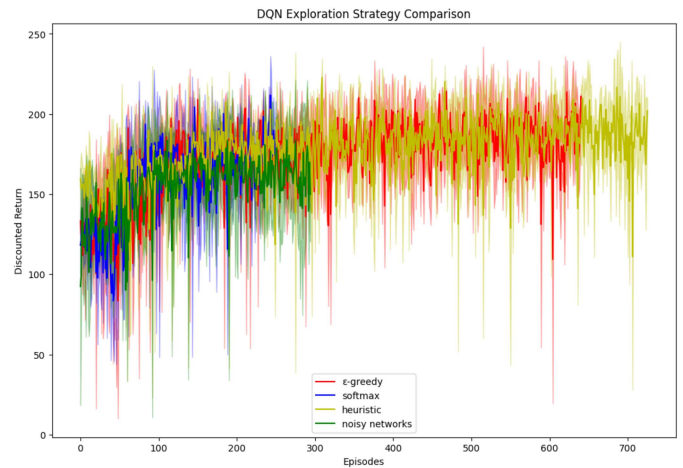
Noisy Networks results

Noisy Network varies significantly between seeds. The average return across all episodes across all seeds was 151.21. The first seed shows a slight curve and then a dip for around 200 episodes, and then a steep learning curve up. The second seed shows a learning curve with a flat line for around 250 episodes. The third seed shows a good start with a sudden dip and then into a typical learning curve, plateauing around 160. The overall learning curve for all seeds can be seen below:

Analysis

The heuristic method was the overall best performer, followed closely by the e-greedy agent. Noisy Network and Softmax agents were the clear underperformers, with Noisy Networks performing significantly the worst. Softmax can be seen to converge the fastest. With more training, this agent may outperform e-greedy. Softmax and Noisy Network both had significantly fewer episodes, meaning that the agent is surviving longer within each episode.

The comparison of all four agents can be seen below:



Limitations

There are a few notable limitations within the experiments:

- Only 3 seeds per agent
- Only 2,000,000 time steps per seed
- Noisy Network could benefit from more tuning
- Only trained on one level

6 Conclusion

Summary

Throughout this paper, we investigated different exploration strategies using DQN in a Super Mario Bros environment.

The four strategies tested were e-greedy, Softmax, heuristic, and Noisy Network. The best strategy was the heuristic method with an average return of 177.64, then e-greedy with an average return of 173.94, followed by Softmax with an average return of 163.92, and lastly Noisy Network with an average return of 151.21.

Key Takeaways

The first key takeaway from these experiments is that having a good knowledge of what should perform well in an environment and incorporating this knowledge into exploration can prove effective. This is what we did for the heuristic method, which performed the best.

The second key takeaway is that more complicated strategies aren't always the best. The Noisy Network agent performed the worst and was the overall most complicated strategy to implement.

Future Work

These experiments could be improved with some potential future work. This would include:

- More seeds per agent
- More time steps per seed
- Noisy Network tuning
- Multiple levels trained on
- Comparison of time-steps rather than episodes

7 Code

The link to the GitHub repo is below:

<https://github.com/doobshub/CS5180-Final-Project>

8 Citations

References

- Fortunato, M., et al. 2017. Noisy networks for exploration.
- Kauten, C. 2018a. gym-super-mario-bros. <https://github.com/Kautenja/gym-super-mario-bros>.
- Kauten, C. 2018b. nes-py: An nes emulator and openai gym interface. <https://github.com/Kautenja/nes-py>.
- Macas, M.; Wu, C.; and Fuertes, W. 2022. A survey on deep learning for cybersecurity: Progress, challenges, and opportunities. *Neurocomputing*.
- Mnih, V., et al. 2015. Human-level control through deep reinforcement learning. *Nature*.
- Pathak, D.; Agrawal, P.; Efros, A. A.; and Darrell, T. 2017. Curiosity-driven exploration by self-supervised prediction. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- Sutton, R. S., and Barto, A. G. 2018. *Reinforcement Learning: An Introduction*. MIT Press.